AI, ML, Deep Learning
Agentic AI & Enterprise Roadmap

Beginner -> Advanced -> Enterprise

12-month practical timeline with projects in every module

Includes Python, Math, pandas, scikit-learn, PyTorch, LLMs, RAG, agents, MLOps and governance

Pace	10-12 hours/week; compress to 6 months at 20+ hours/week
Outcome	Portfolio strong enough for applied ML/AI engineer conversations
Style	Every module ships a working project, not just notes
Final build	One enterprise-grade AI system combining all major technologies

Generated locally by Codex | Updated with current official ecosystem references

Roadmap Overview

Use this as a practical path from fundamentals to production. The sequence is intentional: Python and math first, then data and classical ML, then deep learning, LLM systems, agents, and finally enterprise operations.

1	Weeks 1-4	**Python + tooling**	Syntax, functions, files, OOP basics, venv, Git, notebooks, clean code.
2	Weeks 5-8	**Math for ML**	Linear algebra, calculus intuition, probability, statistics, optimization.
3	Weeks 9-12	**Data stack**	NumPy, pandas, visualization, SQL, EDA, feature quality.
4	Weeks 13-20	**Classical ML**	Supervised/unsupervised ML, sklearn pipelines, validation, metrics.
5	Weeks 21-30	**Deep learning**	PyTorch tensors, autograd, training loops, CNNs, sequence models, tuning.
6	Weeks 31-38	**LLMs + RAG**	Transformers, embeddings, vector search, prompt engineering, evals, safety.
7	Weeks 39-46	**Agentic AI**	Tool use, memory, orchestration, LangChain/LangGraph/OpenAI Agents, MCP basics.
8	Weeks 47-52	**Enterprise AI**	MLOps/LLMOps, MLflow, FastAPI, Docker, CI/CD, monitoring, governance, capstone.

Module Timeline With Projects

1. Module 1: Python Engineering Foundations (2 weeks)

Learn: Python syntax, functions, modules, exceptions, typing, file I/O, virtual environments, Git, pytest basics.
Project: CLI data-cleaning utility that reads messy CSV/JSON, validates schema, logs errors, and writes clean output.
Exit: write small packages, use type hints, run tests, and explain code structure.

2. Module 2: Math For AI/ML (3 weeks)

Learn: vectors/matrices, dot products, derivatives, gradients, probability distributions, Bayes rule, loss functions.
Project: implement linear regression and logistic regression from scratch using only Python/NumPy.
Exit: understand why gradient descent, regularization, and metrics work.

3. Module 3: NumPy, pandas, Visualization, SQL (3 weeks)

Learn: arrays, DataFrames, joins, groupby, time series, missing data, plotting, basic SQL joins/windows.
Project: end-to-end exploratory data analysis report with data quality checks and business recommendations.
Exit: turn raw tabular data into reliable features and clear insights.

4. Module 4: Statistics + Experimentation (2 weeks)

Learn: sampling, confidence intervals, hypothesis tests, A/B tests, leakage, bias, correlation vs causation.
Project: design and analyze an A/B experiment with simulated product metrics.
Exit: avoid common statistical traps in model and product decisions.

5. Module 5: Classical Machine Learning With scikit-learn (5 weeks)

Learn: regression, classification, clustering, preprocessing, pipelines, cross-validation, grid/random search.
Project: customer churn or credit-risk model with sklearn Pipeline, model card, and reproducible notebook.
Exit: choose metrics, compare baselines, prevent leakage, and serialize a model.

Module Timeline With Projects

6. Module 6: Feature Engineering + Model Interpretation (3 weeks)

Learn: categorical encoding, scaling, imbalance, text features, SHAP/permutation importance, fairness checks.
Project: interpretable fraud or anomaly model with feature importance dashboard.
Exit: explain model behavior to technical and non-technical stakeholders.

7. Module 7: PyTorch Deep Learning Foundations (5 weeks)

Learn: tensors, autograd, datasets/dataloaders, training loops, optimizers, schedulers, GPU basics.
Project: image classifier or tabular neural net with custom training loop, checkpointing, and metrics.
Exit: debug loss curves, overfitting, underfitting, device issues, and batch pipelines.

8. Module 8: Computer Vision + NLP Deep Learning (5 weeks)

Learn: CNNs, transfer learning, tokenization, embeddings, sequence models, attention intuition.
Project: document/image classifier plus text sentiment/topic classifier using pretrained backbones.
Exit: use transfer learning pragmatically instead of training large models from scratch.

9. Module 9: Transformers + LLM Foundations (4 weeks)

Learn: transformer architecture, tokenization, embeddings, inference, generation controls, prompt design.
Project: LLM-powered document summarizer with structured outputs and regression test prompts.
Exit: know when to use prompting, RAG, fine-tuning, or classical ML.

Module Timeline With Projects

10. Module 10: Retrieval-Augmented Generation (4 weeks)

Learn: chunking, embeddings, vector DBs, hybrid retrieval, reranking, citations, hallucination controls.
Project: private knowledge-base Q&A bot with source citations, eval dataset, and failure analysis.
Exit: build RAG systems that can be measured and improved.

11. Module 11: Fine-tuning + Evaluation (3 weeks)

Learn: supervised fine-tuning concepts, LoRA/PEFT, preference data, eval design, red teaming, safety checks.
Project: fine-tune or adapter-train a small open model for a domain task and compare against prompting/RAG.
Exit: justify customization with evidence, cost, latency, and risk tradeoffs.

12. Module 12: Agentic AI (4 weeks)

Learn: tools/function calling, planning, memory, multi-step workflows, human-in-the-loop, guardrails.
Project: research assistant agent that searches, reads files, calls tools, writes a report, and logs traces.
Exit: design agents as controlled workflows, not magic loops.

Module Timeline With Projects

13. Module 13: MLOps + LLMOps (4 weeks)

Learn: experiment tracking, model registry, data/version control, MLflow, CI/CD, Docker, FastAPI, monitoring.
Project: deploy a model API with tracking, tests, containerization, rollback plan, and drift dashboard.
Exit: move from notebook to repeatable, observable production system.

14. Module 14: Enterprise AI Architecture (4 weeks)

Learn: security, IAM, secrets, data privacy, audit logs, governance, cost controls, SLAs, incident response.
Project: architecture decision record and risk register for an enterprise AI application.
Exit: discuss AI systems like an engineer accountable for production outcomes.

Skill Checklists

Core Technical Checklist

- [ ] Python package structure, type hints, logging, testing, environment management.
- [ ] Math intuition for gradients, probability, statistics, matrix operations, optimization.
- [ ] pandas/SQL data preparation, joins, time series, missing values, text cleaning, plotting.
- [ ] scikit-learn pipelines, cross-validation, model selection, metrics, model persistence.
- [ ] PyTorch training loops, datasets, dataloaders, GPU/device handling, checkpointing.
- [ ] Transformers, embeddings, RAG, prompt design, structured output, citations, hallucination controls.
- [ ] Agent tool calling, state, memory, workflow orchestration, human approval, observability.
- [ ] MLOps/LLMOps: MLflow, CI/CD, model registry, deployment, monitoring, cost controls.

Topics Often Missed But Needed For Enterprise

- [ ] SQL and data modeling: AI engineers spend more time on data correctness than model code.
- [ ] Software engineering: APIs, tests, Docker, Git workflows, code review, clean architecture.
- [ ] Security: secrets management, least privilege, audit logging, prompt-injection defenses.
- [ ] Evaluation: offline eval sets, golden examples, regression tests, human review, red teaming.
- [ ] Observability: traces, latency, token/cost tracking, drift, quality dashboards, incident playbooks.
- [ ] Governance: model cards, data lineage, approval gates, privacy review, compliance sign-off.
- [ ] Product thinking: user workflows, success metrics, failure modes, escalation paths.

Final Enterprise Capstone

Build one integrated system that demonstrates real-world AI engineering rather than isolated notebooks.

Capstone: Enterprise AI Decision Assistant

Problem: Enterprise AI Decision Assistant for a business domain such as operations, finance, maintenance, HR, or energy.
Data: structured tables, PDFs/docs, logs, and optional image/text data.
ML: pandas feature pipelines, sklearn baseline, PyTorch deep model where justified.
LLM/RAG: document ingestion, embeddings, vector search, citation-backed answers, structured JSON outputs.
Agent: tool-calling workflow that can retrieve docs, query SQL, run model inference, summarize evidence, and ask for human approval.
MLOps/LLMOps: MLflow tracking, eval suite, prompt/model versioning, Dockerized FastAPI service, CI tests, monitoring plan.
Enterprise controls: authentication assumptions, secrets handling, PII policy, rate limits, fallback behavior, audit logs, model card, runbook.

Suggested Architecture

- Frontend or notebook demo -> FastAPI backend -> Auth/session layer -> Agent orchestrator.
- Agent tools -> SQL query tool, document retriever, ML inference endpoint, calculator, report writer.
- Data layer -> PostgreSQL or SQLite, object storage folder, vector database, feature pipeline.
- Model layer -> sklearn baseline, optional PyTorch model, LLM API/open model, embedding model.
- Ops layer -> MLflow tracking, eval suite, Docker, CI tests, logs/traces, cost dashboard.

Final Demo Requirements

- [ ] A user asks a business question and gets an answer with citations, model outputs, and clear confidence/risk notes.
- [ ] The system calls at least three tools: retrieval, SQL/data, and ML inference or calculator.
- [ ] Every answer is logged with prompt/model version, latency, token/cost estimate, and evaluation outcome.
- [ ] There is a fallback path when retrieval is weak or the model is uncertain.
- [ ] A short README explains setup, architecture, risks, and how to operate the system.

Weekly Operating Model

Recommended Weekly Rhythm

2 hours theory: math, algorithms, system design, or papers/docs.
3 hours implementation: code the concept from scratch or with core libraries.
3 hours project work: make a portfolio artifact that runs end-to-end.
1 hour testing/evaluation: add metrics, tests, error analysis, or eval examples.
1 hour writing: README, model card, architecture notes, lessons learned.

Portfolio Milestones

- Month 1: Python CLI package with tests.
- Month 2: math-from-scratch ML notebook.
- Month 3: polished EDA/data quality report.
- Month 5: sklearn production-style pipeline.
- Month 7: PyTorch deep learning project.
- Month 9: RAG app with citations and evals.
- Month 10: tool-using agent with traces.
- Month 12: enterprise capstone with deployment and governance artifacts.

Reference Sources Used

- Python Tutorial: https://docs.python.org/3/tutorial/
- pandas Getting Started: https://pandas.pydata.org/docs/getting_started/index.html
- scikit-learn User Guide: https://scikit-learn.org/stable/user_guide.html
- PyTorch Tutorials: https://docs.pytorch.org/tutorials/
- Hugging Face Transformers: https://huggingface.co/docs/transformers/index
- LangChain / LangGraph docs: https://docs.langchain.com/oss/python/langchain/overview
- OpenAI Agents / Evals / Retrieval / Safety docs: https://platform.openai.com/docs/
- MLflow documentation: https://mlflow.org/docs/latest/index.html